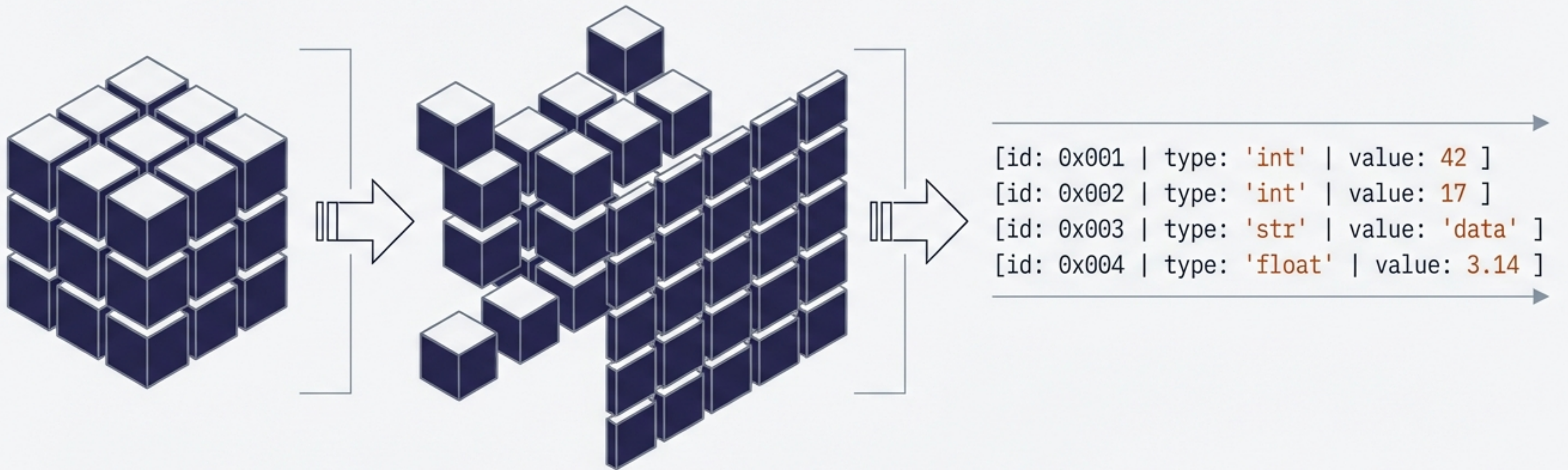


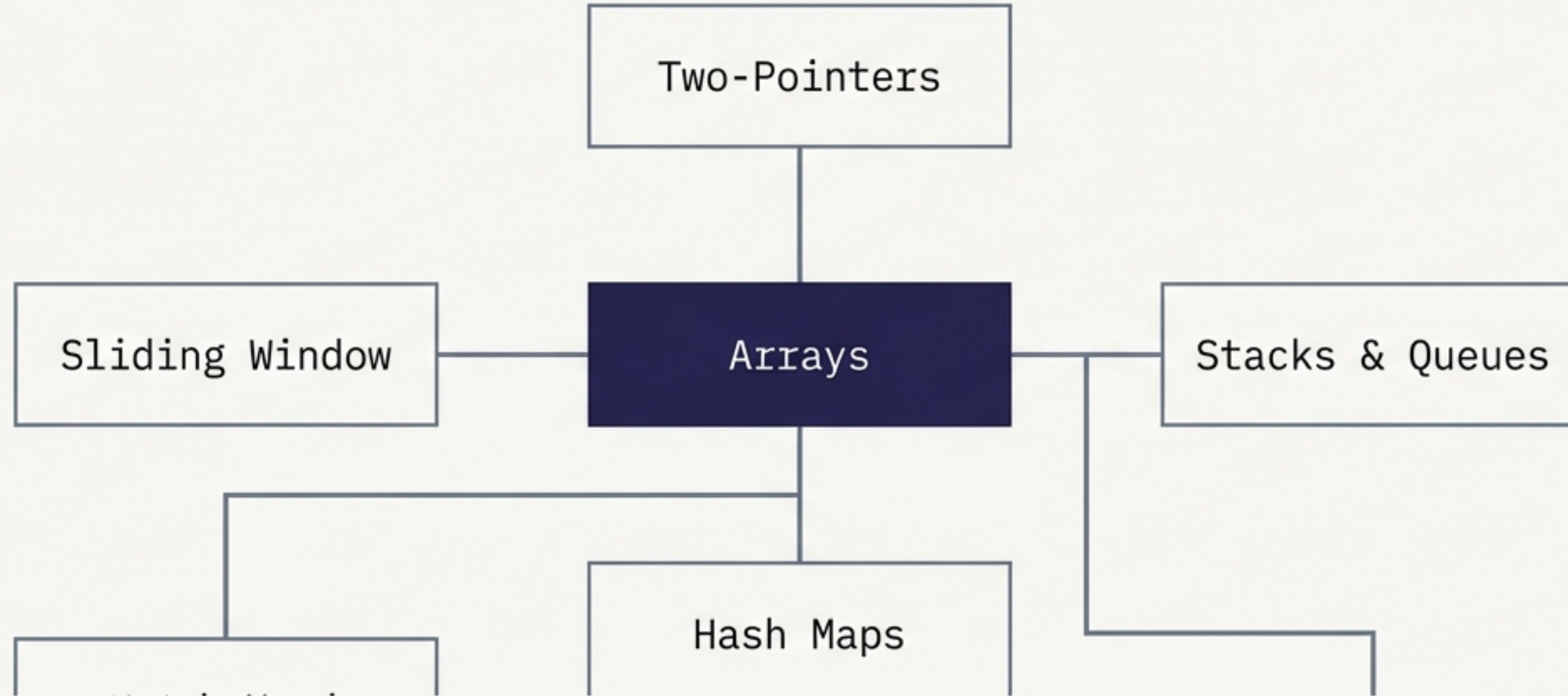
The Blueprint of Python Arrays

Mastering Data Structures: From Memory Blocks to Multi-Dimensional Matrices



// The 90% Foundation of
Data Structures & Algorithms

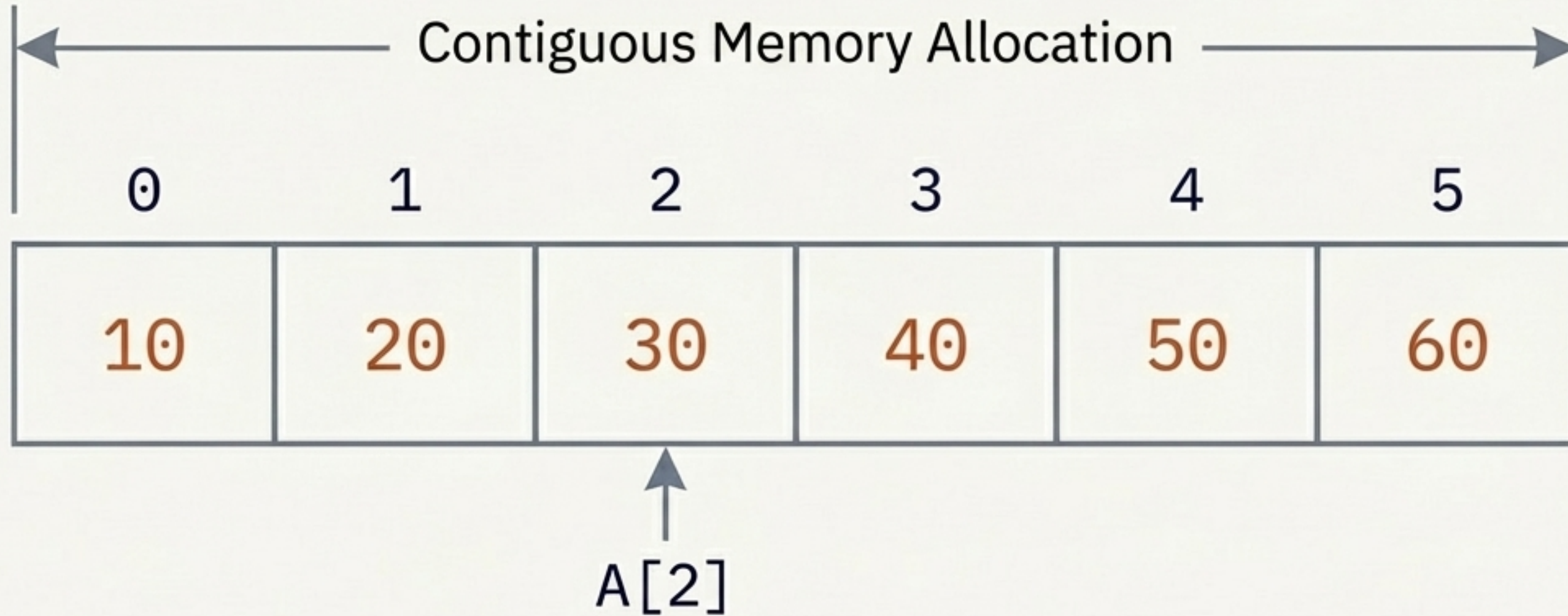
The Core Engine of Algorithmic Logic



90% of Data Structures and Algorithms operations directly touch or rely upon arrays. The DSA story always begins here.

100% vital for real-world application implementation.

The Anatomy of an Array



A systematic data structure allocating continuous memory blocks for rapid element storage and retrieval.

The Golden Rule: Indexing always begins at zero. Element 1 lives at Index 0.

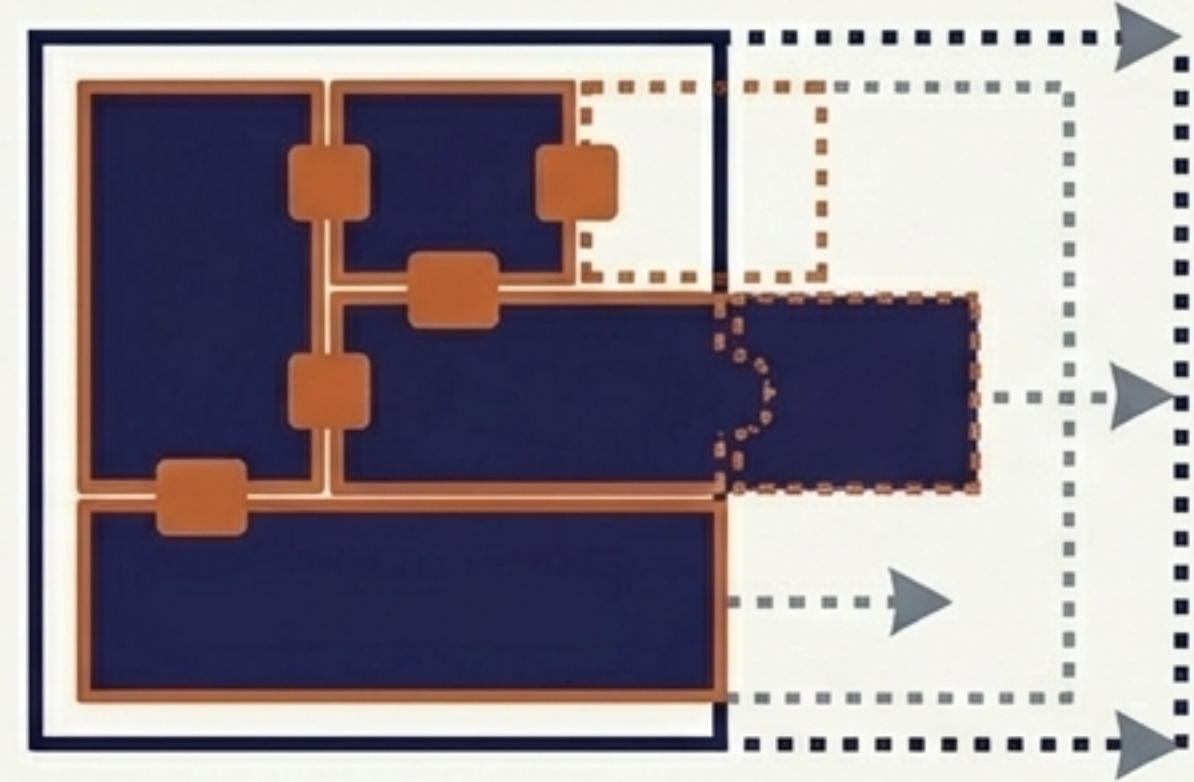
The Python Paradigm Shift

C / C++ / Java



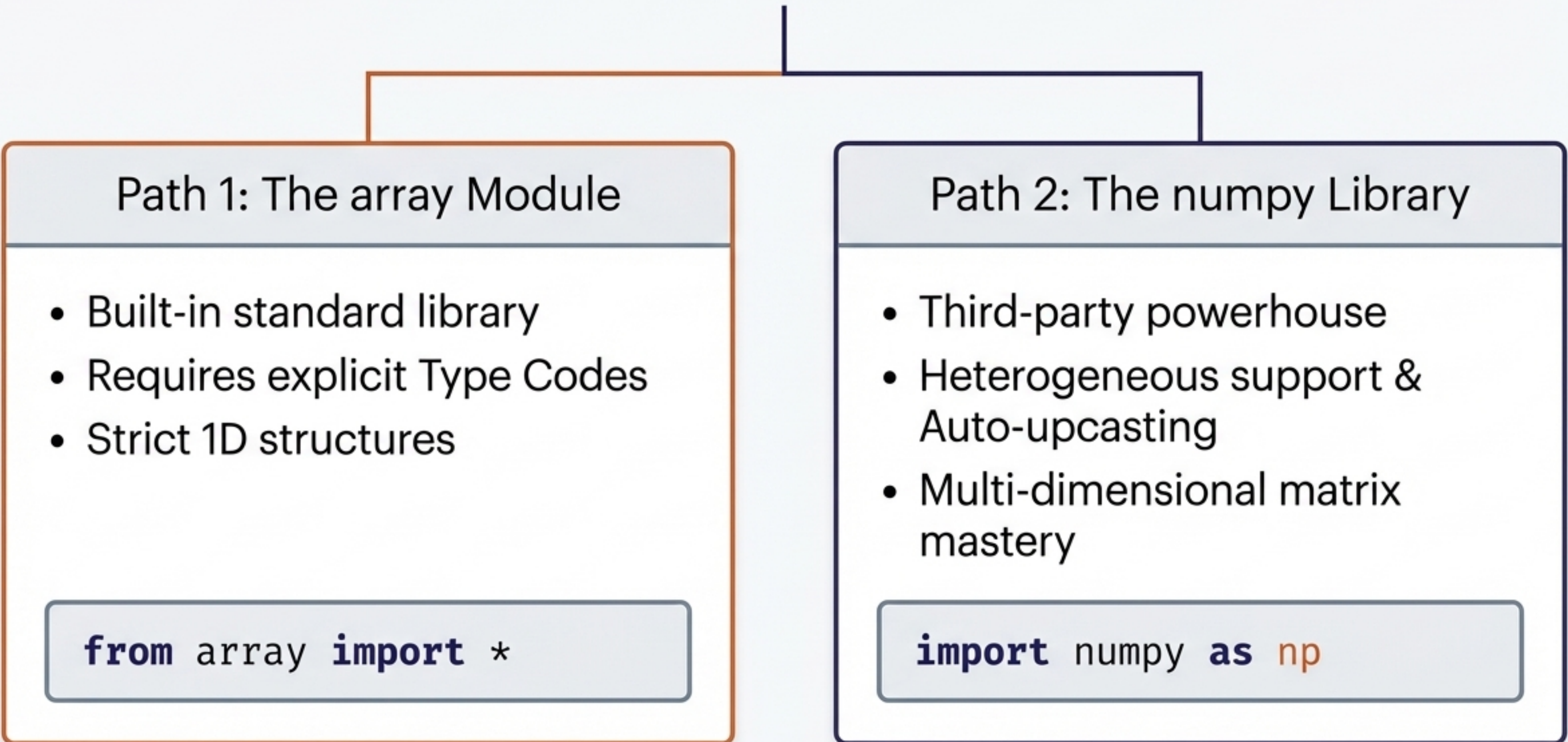
- Memory: Manual / Fixed Size
- Typing: Strict Homogeneous
- Constraint: Size must be declared at creation.

Python



- Memory: Auto-scaling / Dynamic Size
- Typing: Heterogeneous Capabilities
- Freedom: Scales automatically as elements are added.

Two Paths to Python Arrays



Path 1: The array Module

- Built-in standard library
- Requires explicit Type Codes
- Strict 1D structures

```
from array import *
```

Path 2: The numpy Library

- Third-party powerhouse
- Heterogeneous support & Auto-upcasting
- Multi-dimensional matrix mastery

```
import numpy as np
```


Path 1: The Native array Module

```
my_array = array("i", [10, 20, 30, 40])
```



Code	Data Type	Memory Footprint
i / I	Integer	2 Bytes
l / L	Integer	4 Bytes
f / d	Float	4 / 8 Bytes
u	Unicode	Character

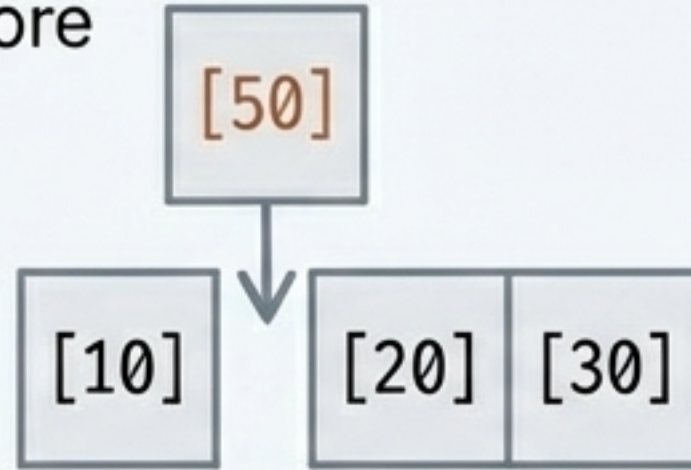
// Unlike Java/C++, Python allows you to specify memory footprint (2-byte vs 4-byte integers) directly via the type code.

Essential Operations: Mutating the Array

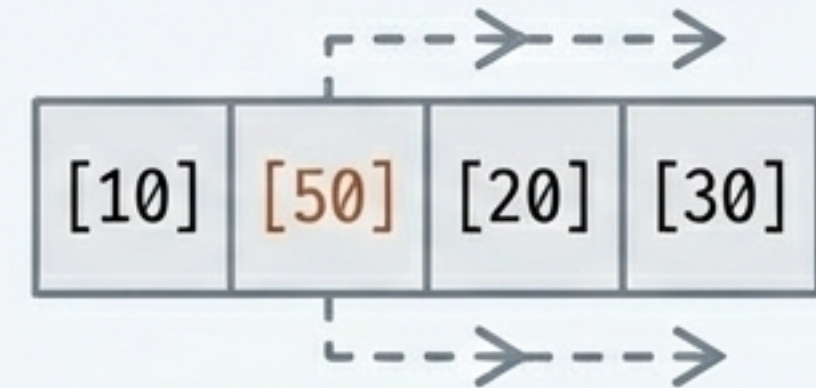
arr.insert(1, 50)

// Shifts elements right.
No deletions.

Before



After



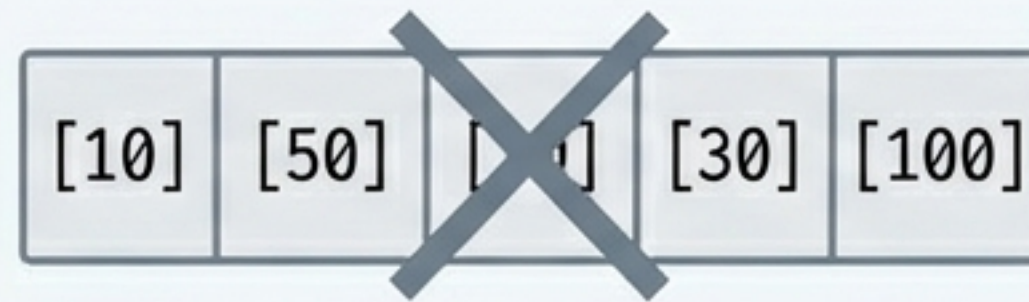
arr.append(100)

// Attaches a new block
to the far right.



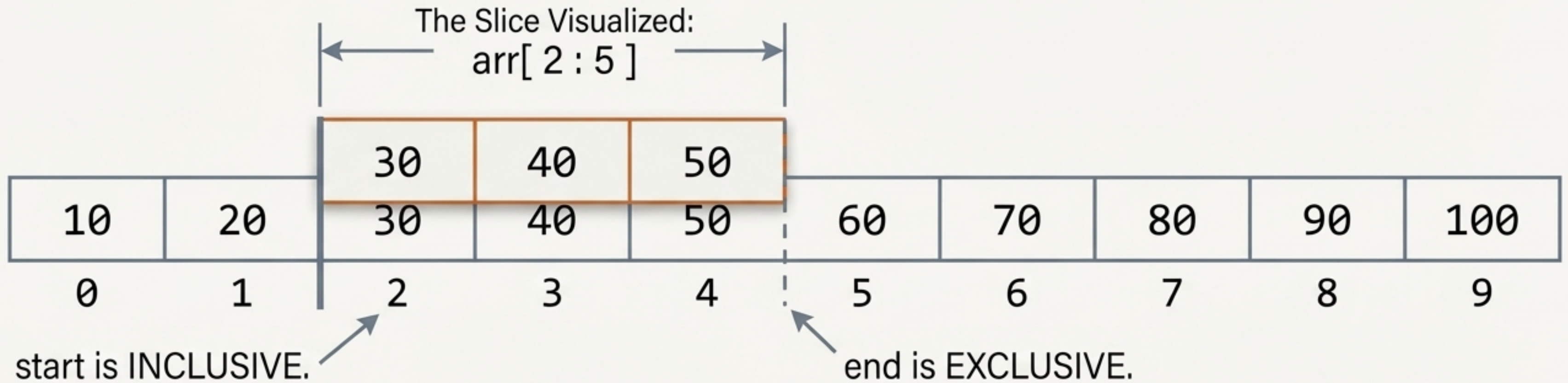
arr[2] = 200

// Direct overwrite.
No shifting.



Array Slicing Mastery

```
arr[ start : end : step ]
```



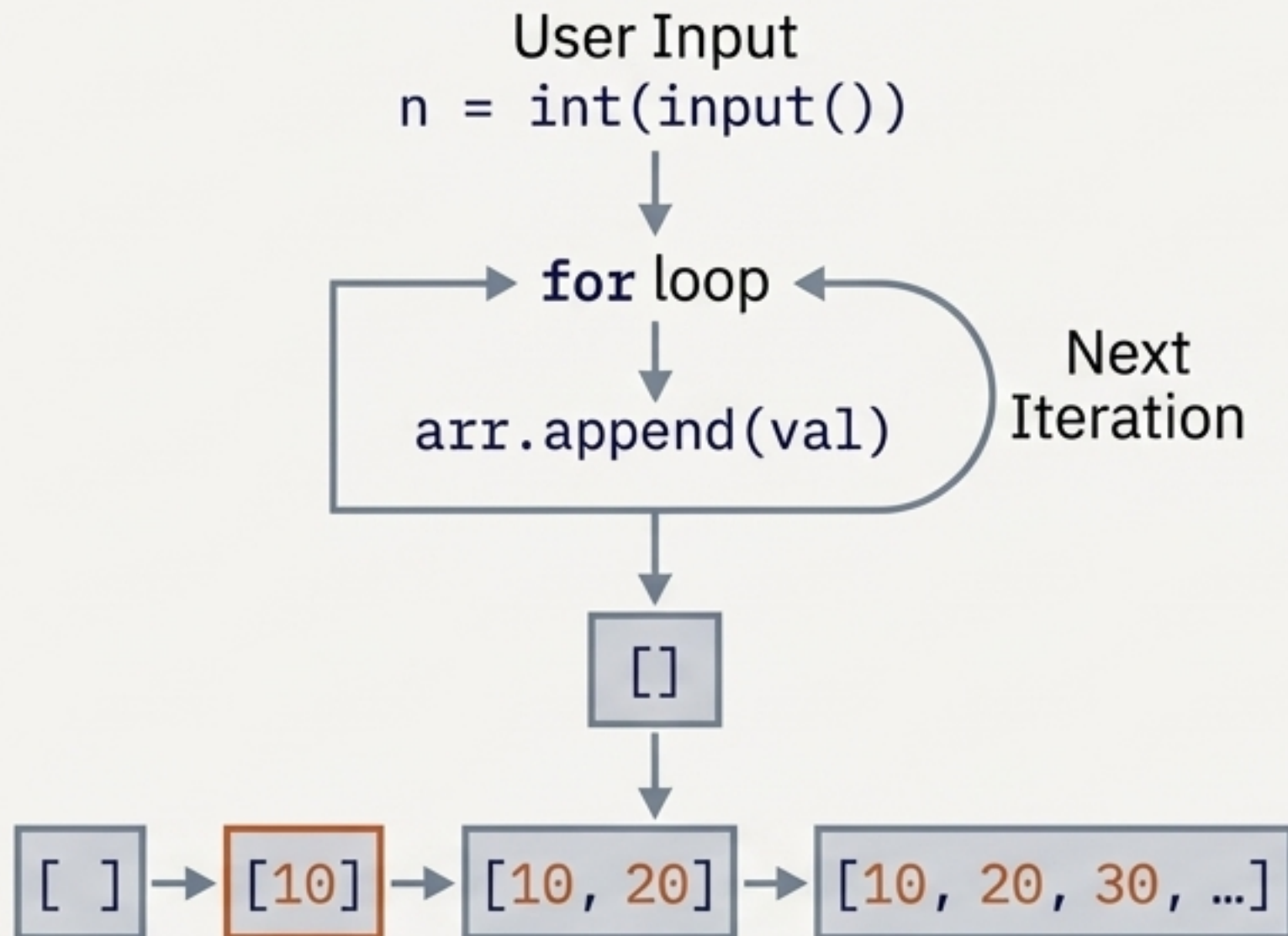
The Magic Trick

```
arr[ ::-1 ]
```

Instant complete array reversal. Eliminates the need for manual swapping loops used in **C++/Java**.

Dynamic Construction & Searching

Dynamic Building



Arrays start empty and scale dynamically based on run-time user input.

Searching via `index()`

`arr.index(134)`

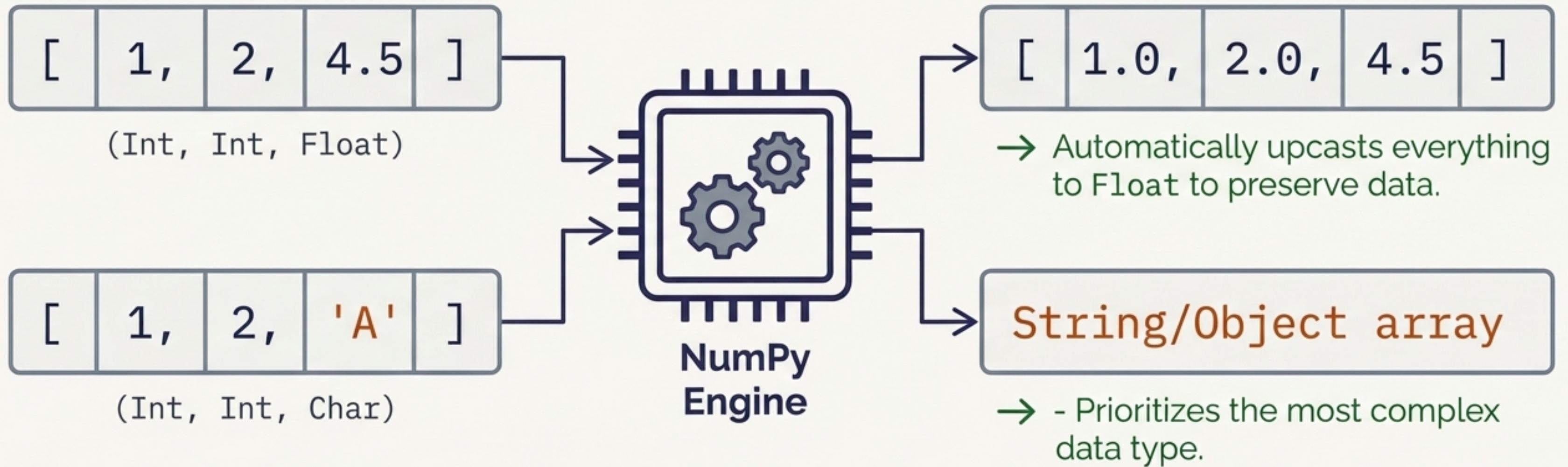


Instantly returns the location (e.g., Index 2).

Throws an exception if the element does not exist in the array.

Path 2: Enter numpy (The Power Upgrade)

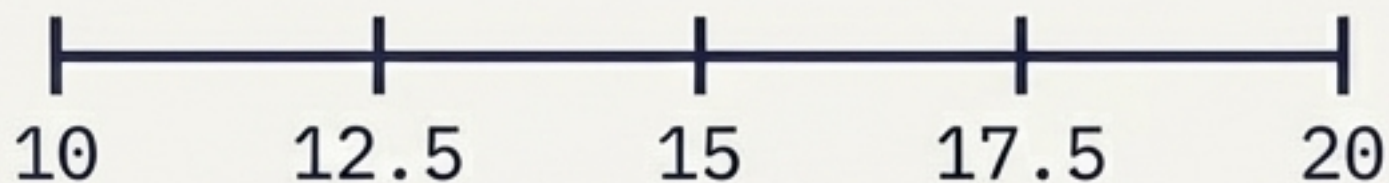
No typecodes required. Welcome to heterogeneous data handling.



Explicit control: You can force types using `dtype=float`.

The numpy Generator Toolkit

`linspace(10, 20, 5)`



5 equal spatial divisions.
End value (20) IS included.

`arange(10, 20, 2)`



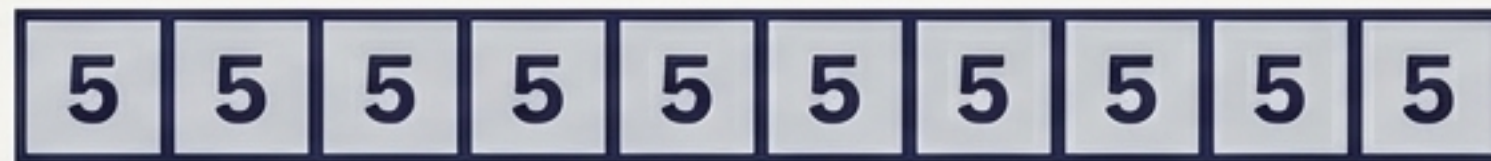
Steps of 2. End value (20) IS EXCLUDED.
[10, 12, 14, 16, 18].

`zeros(10) / ones(10)`



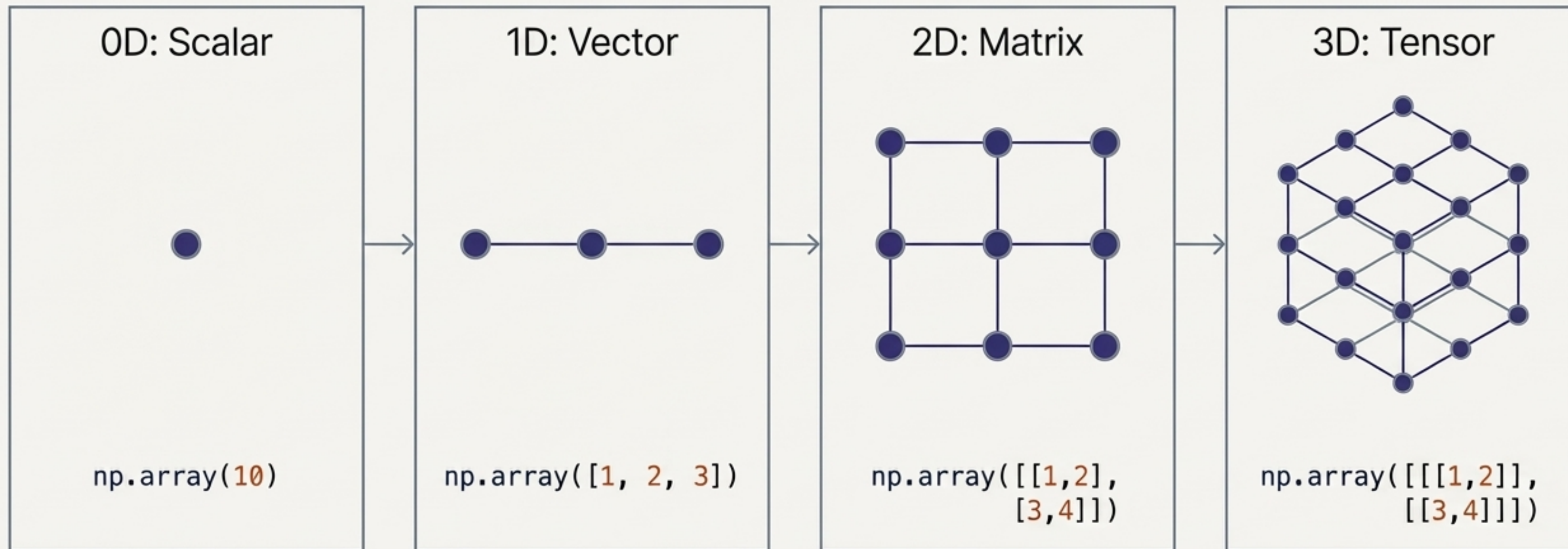
Array filled entirely with 0s or 1s.
Ideal for boolean/state tracking.

`full(10, 5)`



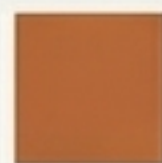
An array of size 10 entirely populated
with the number 5.

Visualizing Dimensionality

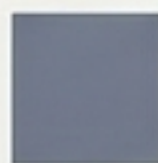


Syntax of the Multiverse

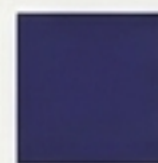
[[[1, 2], [3, 4]],
[[5, 6], [7, 8]]]



The 1D Rows
(Elements)



The 2D Matrices
(Collections of 1D
rows)



The 3D Tensor
(The master container
of 2D grids)

Insight

A 3D array is simply
an array of 2D arrays.

Synthesis: Choosing Your Array Structure

Technology	Best For	Dimensions	Type Strictness
Python Standard Lists <code>[]</code>	General scripting, fast prototyping.	1D (primarily).	None (fully dynamic).
<code>array</code> Module	Strict memory management, lightweight C-like operations.	1D strictly.	High (Requires Type Codes).
<code>numpy</code> Arrays	Data science, mathematics, complex modeling.	N-Dimensional (0D to 3D+).	Flexible (Auto-upcasting / Heterogeneous).

Let the problem define the architecture.
Master all three to master Python Data Structures.